



รายงาน

การศึกษาและออกแบบ Algorithm สำหรับการแก้ไขปัญห Rastrigin Function

ในการหา Global Minimum

รายวิชา

FRA502 Advanced Robotic Control and Simulation with ROS2 (ARCS2)

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ประจำปีการศึกษา 1/2568

ผู้จัดทำ

นางสาวดิษย์ธร สุทธาเวช รหัสนักศึกษา 66340500019

นางสาวบุญยาพร ปรีชาศุทธิ์ รหัสนักศึกษา 66340500031

Introduction

การหาค่าเหมาะที่สุดทั่วโลก (Global Optimization) เป็นปัญหาพื้นฐาน ที่สำคัญในหลากหลายสาขาวิชา โดยเฉพาะอย่างยิ่งในกรณีที่ฟังก์ชันวัตถุประสงค์มีลักษณะซับซ้อนและเป็น Non-Convex ซึ่งมีค่าต่ำสุดเฉพาะที่ (Local Minima) กระจายอยู่หลายจุด การค้นหาค่าต่ำสุดทั่วโลก (Global Minimum) จึงเป็นความท้าทายแบบหนึ่ง

Rastrigin Function ถือเป็นฟังก์ชันมาตรฐานที่นิยมใช้ในการทดสอบประสิทธิภาพของอัลกอริทึมในการหาค่าเหมาะที่สุด เนื่องจากมีโครงสร้างที่ซับซ้อนจากการมี Local Minima จำนวนมาก ทำให้วิธีการแบบดั้งเดิมที่อาศัยการคำนวณ Gradient มักติดอยู่ใน Local Minima และไม่สามารถค้นหา Global Minimum ได้ ฟังก์ชันนี้มีรูปแบบดังสมการที่ (1)

$$f(x) = An + \sum_{i=1}^n (x_i^2 - A \cos(2\pi x_i)) \quad (1)$$

$$X = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$$

$$A = 10 \text{ (ค่าคงที่)}$$

$$n = \text{จำนวนมิติ (dimensions)}$$

กรณี 2 มิติ (2D Case/ 2 DOF)

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10 \cos(2\pi x_1) - 10 \cos(2\pi x_2) \quad (2)$$

Global Minimum อยู่ที่

$$x^* = (0, 0, \dots, 0), f(x^*) = 0$$

ในขอบเขตของการหาค่าเหมาะที่สุดแบบดั้งเดิม มีแนวคิดพื้นฐาน 2 ประเภทหลัก 1) การหาค่าเหมาะที่สุดแบบไม่มีเงื่อนไขบังคับ (Unconstrained Optimization) เช่น Newton's Method และ Damped Newton's Method ที่ปรับปรุงความเสถียรของการลู่เข้าสู่คำตอบ 2) การหาค่าเหมาะที่สุดแบบมีเงื่อนไขบังคับ (Constrained Optimization) เช่น Quadratic Programming และ Augmented Lagrangian Method ที่สามารถจัดการกับข้อจำกัดต่าง ๆ ได้ ซึ่งวิธีการเหล่านี้ถูกออกแบบมาสำหรับปัญหาที่มีลักษณะ Convex หรือมีความราบรื่นเพียงพอ ทำให้เมื่อนำไปประยุกต์ใช้กับปัญหา Non-Convex เช่น Rastrigin Function มักจะประสบปัญหาการติดอยู่ใน Local Minima และไม่สามารถรับประกันการหลีกเลี่ยง Local Minima ได้อย่างสมบูรณ์

เนื่องจากข้อจำกัดของวิธีการแบบดั้งเดิม จึงมีการพัฒนาวิธีการหาค่า Global Minimum ที่ออกแบบมาโดยเฉพาะสำหรับปัญหา Non-Convex ซึ่งสามารถแบ่งออกเป็น 2 Method ได้แก่ 1) วิธีการเชิงกำหนด (Deterministic Methods) และ 2) วิธีการแบบสุ่ม (Stochastic Methods) ซึ่ง Deterministic Methods มีความสามารถในการรับประกันการค้นพบ Global Minimum ภายในความคลาดเคลื่อนที่กำหนดไว้ (ϵ) โดยอาศัยการสำรวจพื้นที่คำตอบอย่างเป็นระบบ ในขณะที่ Stochastic Methods ใช้กลไกการสุ่มในการสำรวจพื้นที่ คำตอบ ทำให้มีความรวดเร็วและสามารถประยุกต์ใช้กับปัญหาที่มีมิติสูงได้ แต่ไม่สามารถรับประกัน Global Optimality ได้

การศึกษานี้จึงมุ่งเน้นการเปรียบเทียบทั้งสองแนวทาง ได้แก่ อัลกอริทึม Branch and Bound (BnB) จาก Deterministic Methods และ อัลกอริทึม Simulated Annealing (SA) จาก Stochastic Methods เพื่อวิเคราะห์ Trade-off ระหว่างการรับประกันความถูกต้องกับประสิทธิภาพในการคำนวณ

Deterministic Algorithm

Branch and Bound (BnB) เป็นวิธี Deterministic ที่ใช้หลักการ Divide and Conquer แบ่งพื้นที่คำตอบออกเป็น Box ย่อย ๆ (B) และคำนวณ Lower Bound (LB) กับ Upper Bound (UB) เพื่อตัดทิ้ง Box ที่ไม่มี Global Minimum วิธีนี้รับประกันการพบ Global Minimum ที่อยู่ในขอบเขต (domain) ที่กำหนดทางคณิตศาสตร์ โดยคำตอบจะอยู่ภายในความคลาดเคลื่อน ϵ ที่กำหนด

Lower Bound สำหรับ $box\ B = [a_1, b_1] \times [a_2, b_2]$

$$LB(B) = An + \sum_{i=1}^n (i-1) [\min a_i^2, b_i^2 \cdot 0 - A] \quad (3)$$

Upper Bound (ประเมินที่จุดกลาง)

$$UB(B) = f\left(\frac{a+b}{2}\right) \quad (4)$$

Pruning condition (เงื่อนไขตัดทิ้ง Box)

$$LB(B) \geq f_{best} \quad (5)$$

Stopping criterion (เงื่อนไขหยุดแบ่ง box)

$$\max_{(i=1, \dots, n)} (b_i - a_i) < \epsilon \quad (6)$$

อัลกอริทึม BnB ประกอบด้วยสามแนวคิดหลักที่ทำงานร่วมกันของ

- 1) Branching เป็นกระบวนการแบ่ง Box ออกเป็นสองส่วนย่อยตามมิติที่มีความกว้างมากที่สุด โดยแบ่งที่จุดกึ่งกลาง การเลือกมิติที่กว้างที่สุดช่วยให้ Box มีขนาดเล็กลงอย่างรวดเร็ว ซึ่งเป็นเงื่อนไขหยุดของ Algorithm
- 2) Bounding เป็นกระบวนการคำนวณขอบเขตล่าง (LB) ซึ่งรับประกันว่า $f(x) \geq LB$ สำหรับทุกจุด x ใน Box และขอบเขตบน (UB) ที่ได้จากการประเมินค่าฟังก์ชันที่จุดกึ่งกลางของ Box การคำนวณ LB ที่แม่นยำเป็นกุญแจสำคัญต่อประสิทธิภาพของ BnB
- 3) Pruning เป็นกลไกตัดทิ้ง Box ที่มี LB มากกว่าหรือเท่ากับค่าที่ดีที่สุดที่พบจนถึงปัจจุบัน (f_{best}) หรือก็คือการตัด Box ที่มั่นใจว่าไม่มี Global Minimum ทำให้ช่วยลดจำนวน Box ที่ต้องสำรวจลงได้

กระบวนการทำงานของ BnB เริ่มต้นด้วยการคำนวณ Lower bound และ Upper bound ของ Domain เริ่มต้น จากนั้นเข้าสู่วงจรหลักที่ทำงานซ้ำจนกว่าจะหยุด 1) เลือก Box ที่มี Lower bound ต่ำที่สุดจาก Priority queue 2) ตรวจสอบเงื่อนไขหยุด หากความกว้างสูงสุดของ Box น้อยกว่า ϵ ให้บันทึกคำตอบและหยุดแบ่ง Box นั้น 3) มิฉะนั้นให้ทำ Branching แบ่ง Box ออกเป็นสองส่วนตามมิติที่กว้างที่สุด 4) คำนวณ LB และ UB ของแต่ละ Box ย่อย โดย UB คำนวณจากการประเมินค่าฟังก์ชันที่จุดกึ่งกลางและทุกมุม (2^d จุด) 5) ทำ Pruning โดยตัดทิ้ง Box ที่มี $LB \geq f_{best}$ เนื่องจากมั่นใจว่าไม่มี Global Minimum และ 6) เพิ่ม Box ที่เหลือเข้า Queue เพื่อสำรวจต่อ

Stochastic Algorithm

Simulated Annealing (SA) เป็นวิธี Stochastic ที่ได้แรงบันดาลใจจากกระบวนการ ชุบโลหะ (Annealing) ในทางโลหวิทยา ซึ่งเป็นการให้ความร้อนกับโลหะแล้วค่อยๆ ลดอุณหภูมิลงอย่างช้าๆ เพื่อให้โครงสร้างผลึกเรียงตัวอยู่ในสถานะที่มีพลังงานต่ำที่สุด อัลกอริทึม SA นำหลักการนี้มาประยุกต์ใช้กับปัญหา Optimization โดยใช้ Temperature Parameter (T) ในการควบคุมความน่าจะเป็นในการยอมรับคำตอบที่แย่ลง ซึ่งช่วยให้สามารถหลุดพ้นจาก Local Minima ได้

SA จะเริ่มต้นจากจุดเริ่มต้นแบบสุ่ม (x_0) และทำการสำรวจพื้นที่คำตอบโดย การสร้าง Neighbor Solution (x') ซึ่งอยู่ใกล้เคียงกับคำตอบปัจจุบัน (x) การยอมรับ คำตอบใหม่ขึ้นอยู่กับ

- 1) หาก $f(x') < f(x) \rightarrow$ ยอมรับเสมอ (คำตอบดีขึ้น)
- 2) หาก $f(x') \geq f(x) \rightarrow$ ยอมรับด้วยความน่าจะเป็นตาม Metropolis Criterion

Acceptance Probability (Metropolis Criterion)

$$P(\text{accept}) = \{1 \text{ if } \Delta f < 0, e^{(-\Delta f/T)} \text{ if } \Delta f \geq 0\} \quad (7)$$

โดยที่

$$\Delta f = f(x') - f(x) \text{ (ความแตกต่างของค่า objective function)}$$

T = Temperature ณ iteration ปัจจุบัน

อุณหภูมิจะค่อยๆ ลดลงตามจำนวน iteration โดยใช้ Linear Cooling Schedule

$$T(k) = T_0(1 - k/k_{max}) \quad (8)$$

การสร้าง Neighbor Solution ใช้การเคลื่อนที่แบบสุ่มรอบๆ จุดปัจจุบัน

$$x'_i = x_i + \text{Uniform}(-s_i, s_i) \quad (9)$$

$$s_i = 0.5 \times (b_i - a_i) \times T(k)/T_0 \quad (10)$$

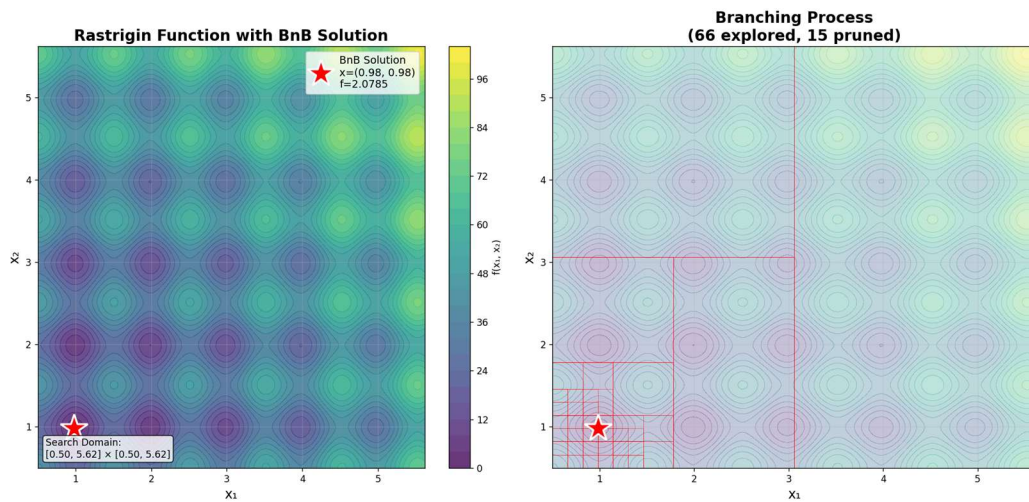
โดยที่

- s_i = Step Size สำหรับมิติที่ i
- $[a_i, b_i]$ = ขอบเขตของตัวแปร x_i
- Step Size จะลดลงตาม Temperature เพื่อให้การค้นหาละเอียดขึ้นเมื่อใกล้จบ

กระบวนการทำงานของ SA เริ่มต้นด้วยการสุ่มจุดเริ่มต้น x_0 ในพื้นที่คำตอบและตั้งค่า Initial temperature T_0 จากนั้นเข้าสู่วงจรหลักที่ทำซ้ำ k_{max} ครั้ง 1) คำนวณ Temperature ปัจจุบัน $T(k)$ ตาม Cooling schedule ที่กำหนดใช้ Linear cooling 2) สร้าง Neighbor solution x' โดยการเคลื่อนที่แบบสุ่มรอบๆ จุดปัจจุบัน x ด้วย Step Size ที่ลดลงตาม Temperature เพื่อให้การสำรวจหาบในวงแรกและละเอียดในช่วงท้าย 3) คำนวณความแตกต่าง Δf 4) ตัดสินใจยอมรับหรือปฏิเสธ x' ตาม Metropolis criterion ซึ่งทำให้สามารถหลุดพ้นจาก Local minima ได้ในช่วงที่ Temperature สูง 5) อัปเดต Current Solution ถ้ายอมรับ และอัปเดต Best Solution ถ้าพบคำตอบที่ดีกว่า

Results and Analysis

ในการทดลองได้เลือกใช้ Offset domain $[0.5, 5.62]^d$ แทน Standard domain $[-5.12, 5.12]^d$ โดยรักษาสีขนาด Domain ไว้เท่าเดิม (5.12 หน่วย) การออกแบบนี้มีวัตถุประสงค์เพื่อ 1) ทำให้ Global minimum ที่แท้จริง $x^* = (0, 0, \dots, 0)$ อยู่นอกขอบเขตแต่อยู่ใกล้ Boundary (ห่าง 0.5 หน่วย) สะท้อนสถานการณ์จริงที่ Optimum อยู่ใกล้ข้อจำกัดทางกายภาพ 2) Best Solution ใน Domain อยู่ที่มีมุม $x^* \approx (0.5, 0.5, \dots, 0.5)$ บังคับให้ BnB ต้องตรวจสอบทุก Corner (2^d points) และทดสอบว่า SA สามารถสำรวจ Boundary ได้ดีเพียงใด 3) ระยะห่าง 0.5 หน่วยไม่ไกลจนเกินไปไม่ได้ แต่ไม่ใกล้จนกลายเป็นปัญหาง่าย ทำให้สามารถแยกแยะความสามารถของแต่ละอัลกอริทึมได้ชัดเจน Domain Design นี้จึงสร้างสถานการณ์ทดสอบที่ท้าทายและสมจริง เหมาะสำหรับการประเมิน Trade-off ระหว่าง Guarantee กับ Scalability ในปัญหาที่มี Boundary constraints

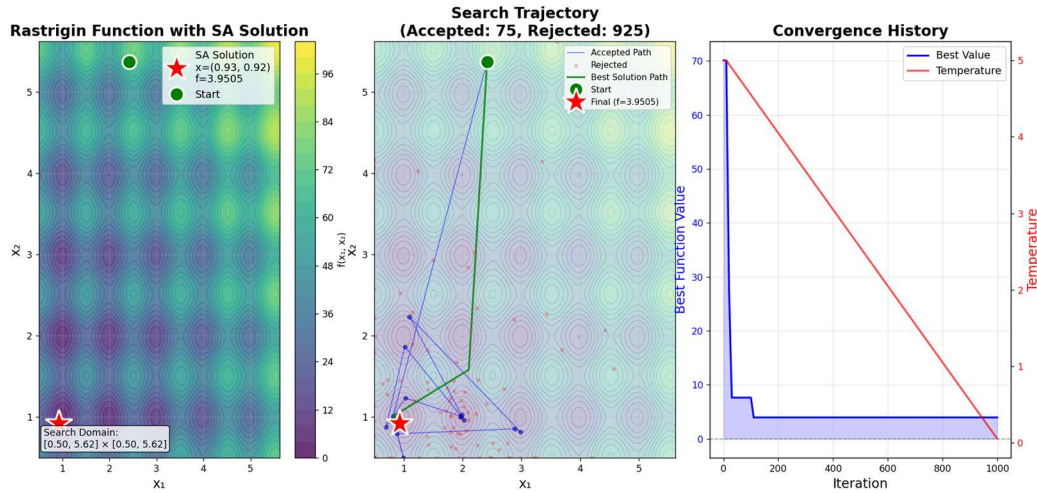


รูปที่ 1 Rastrigin Function with BnB Solution of bounds $[0.5, 5.62]^2$, $\epsilon = 0.2$

ตารางที่ 1 ผลลัพธ์การหา Global Minimum ด้วย Branch and Bound

Domain	ϵ	Solution	$f(x^*)$	Time (s)
2D Offset Bounds $[0.5, 5.62]^2$	0.40	(0.50, 0.50)	2.079	0.003
	0.20	(0.50, 0.50)	2.079	0.005
	0.10	(0.50, 0.50)	2.079	0.019
	0.05	(0.50, 0.50)	2.000	0.049
3D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	3.118	0.053
4D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	4.157	0.695
5D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	5.196	10.332

จากผลการทดลองใน Domain $[0.5, 5.62]^d$ พบว่า Branch and Bound (BnB) สามารถหาคำตอบที่ดีที่สุดได้อย่างสม่ำเสมอ โดยมีค่า $f(x^*) = 2.079, 3.118, 4.157$, และ 5.196 สำหรับ 2D, 3D, 4D, และ 5D ตามลำดับ ซึ่งเป็นคำตอบที่ดีที่สุดที่เป็นไปได้ในขอบเขตที่กำหนด ในด้านเวลาการทำงาน BnB แสดงความเหนือกว่าใน 2D ด้วย $\varepsilon = 0.2-0.4$ ที่ใช้เวลาเพียง 0.003-0.019 วินาที เร็วกว่า SA ถึง 4-6 เท่า อย่างไรก็ตาม เมื่อมิติเพิ่มขึ้นเป็น 5D BnB ใช้เวลา 10.332 วินาที ซึ่งช้ากว่า SA ถึง 213 เท่า แสดงให้เห็น Exponential time complexity $O((L/\varepsilon)^d)$ ที่เป็นข้อจำกัดสำคัญ



รูปที่ 2 Rastrigin Function with SA Solution bounds $[0.5, 5.62]^2$

ตารางที่ 2 ผลลัพธ์การหา Global Minimum ด้วย Simulated Annealing

Domain	Solution	$f(x^*)$	Time (s)
2D Offset Bounds $[0.5, 5.62]^2$	(0.93, 0.92)	3.951	0.019
	(0.93, 0.92)	3.951	0.022
	(0.93, 0.92)	3.951	0.018
	(0.93, 0.92)	3.951	0.019
3D Offset	$\sim(1.0, 1.0, 1.0)$	3.010	0.028
4D Offset	$\sim(1.0, 1.0, \dots)$	7.029	0.038
5D Offset	$\sim(0.5, 0.5, \dots)$	16.942	0.048

ในทางกลับกัน SA แสดงประสิทธิภาพด้านความเร็วและ Scalability ด้วย Linear time complexity $O(d \times k)$ โดยใช้เวลาเพียง 0.019-0.048 วินาทีในทุกมิติ และเวลาเพิ่มขึ้นเพียง ~ 0.01 วินาทีต่อมิติ อย่างไรก็ตาม SA มีข้อจำกัดสำคัญด้านความแม่นยำ โดยให้คำตอบที่แย่กว่า BnB ในทุกมิติ: ใน 2D มีค่า $f(x^*) = 3.951$ (แย่กว่า BnB 90%) และใน 5D ได้ $f(x^*) = 16.942$ (แย่กว่า BnB 226%) สาเหตุหลักมาจาก Search space ที่ขยายตัวแบบ Exponential แต่จำนวน Iterations คงที่ ทำให้การสำรวจไม่ครบถ้วน โดยเฉพาะบริเวณ Boundary ที่ Optimal solution มักอยู่

Conclusion

การเปรียบเทียบแสดงให้เห็น Trade-off ที่ชัดเจนระหว่าง Guarantee กับ Scalability ซึ่ง BnB เหมาะสำหรับปัญหามิติต่ำที่ต้องการความแม่นยำสูง โดยรับประกันการพบ Global Minimum และให้ผลลัพธ์ที่แม่นยำกว่า SA ในมิติสูง แต่มีข้อจำกัดด้าน Scalability จาก Exponential time complexity

นอกจากนี้ BnB ยังต้องการหน่วยความจำสูงสำหรับเก็บ Queue ของ Boxes และต้องคำนวณ 2^d corners ต่อ Box ซึ่งทำให้ไม่สามารถใช้กับปัญหา 5D ขึ้นไปได้อย่างมีประสิทธิภาพ ในทางกลับกัน SA แสดงความเหนือกว่าด้านความเร็วและ Scalability โดยเฉพาะในมิติสูง ที่เร็วกว่า BnB และมี Linear time complexity ที่คาดการณ์ได้ ทั้งยังใช้หน่วยความจำต่ำและ Implement ง่ายด้วย Parameter เพียง 2 ตัว อย่างไรก็ตาม SA มีจุดอ่อนสำคัญด้าน Accuracy ที่ลดลงอย่างมีนัยสำคัญเมื่อมิติเพิ่มขึ้น และไม่รับประกันการพบ Global Minimum ซึ่งอาจไม่เหมาะสมสำหรับ Critical applications

จากผลการวิเคราะห์ สามารถสรุปแนวทางการเลือกใช้อัลกอริทึมได้ตามลักษณะปัญหาดังนี้
เลือกใช้ Branch and Bound เมื่อ

- ปัญหามีจำนวนมิติไม่เกิน 4 มิติ และยอมรับเวลาการทำงานไม่เกิน 1 วินาที
- ต้องการความแม่นยำสูงสุดหรือต้องการรับประกันการพบ Global Minimum ที่แท้จริง
- เป็น Safety-critical Application ที่ไม่สามารถยอมรับคำตอบที่ไม่เหมาะสมได้ เช่น การหลีกเลี่ยง Singularity Configuration ของหุ่นยนต์
- มี Computational Resource เพียงพอสำหรับการเก็บและประมวลผล Queue
- ค่า ϵ ที่ยอมรับได้ไม่เล็กเกินไป ($\epsilon \geq 0.2$) เพื่อให้ได้เวลาที่สมเหตุสมผล
- ปัญหามีลักษณะที่ Lower Bound สามารถคำนวณได้แม่นยำ ทำให้ Pruning มีประสิทธิภาพ

เลือกใช้ Simulated Annealing เมื่อ

- ปัญหามีจำนวนมิติมากกว่า 4 มิติ ซึ่ง BnB จะช้าเกินไป (>10 วินาที)
- ต้องการความเร็วในการตอบสนอง โดยเฉพาะ Real-time Applications ที่ต้องการคำตอบภายใน <0.1 วินาที
- ยอมรับคำตอบที่ดีเพียงพอ (Good Enough Solution) แทนที่จะเป็น Perfect Solution โดย $\text{Error} \leq 50\%$ ยังยอมรับได้
- มีข้อจำกัดด้าน Memory และต้องการอัลกอริทึมที่ใช้ Memory น้อย
- ต้องการ Implementation ที่ง่ายและรวดเร็ว โดยไม่ต้องออกแบบ Bounding Functions ที่ซับซ้อน
- สามารถรัน SA หลายครั้ง (Multi-start) เพื่อเพิ่มโอกาสหาคำตอบที่ดีกว่า

สำหรับการประยุกต์ใช้กับ Motion Planning ของหุ่นยนต์ ผลการศึกษานี้แนะนำให้ใช้ BnB สำหรับหุ่นยนต์ 2-4 DOF ที่ต้องการหลีกเลี่ยง Singularity Configuration อย่างแน่นอน และใช้ SA (พร้อมการปรับปรุง) สำหรับหุ่นยนต์ 6-7 DOF ที่ต้องการความเร็วในการตอบสนองแบบ Real-time โดยอาจต้องใช้ Hybrid approach ที่รวม SA สำหรับ Global search กับ Local optimization สำหรับ Refinement เพื่อให้ได้ Balance ระหว่างความเร็วและความแม่นยำ